# Don’t trust completion. Verify it.

## Coding Agent Harness

An AI assistant edits files and runs tests.
Ask it to change two files. It says “done.”
What changed, and did the checks pass?

**WORKER CLAIM**

“Two files changed.
Tests passed.”

**HARNESS EVIDENCE**

Observed Git changes
Independently run checks

**Human review decides whether to accept**

Experimental Python project. Extensive AI-assisted development and review.

# Architecture and trust boundaries

**CALLER** Task + scope + checks + clean baseline

**Worker process**
QwenWorker / worker.py

→

**Git observations**
git.py • after worker exits

→

**Configured checks**
verifier.py + process.py

Structured completion claim

Paths, patch, HEAD, commits

Exit, output, timeout

**run**
orchestrator.py

One invocation
Mechanical findings

→

Readiness or attention needed
Optional Markdown report

**control**
HarnessController
+ HumanReviewer

Human request
Freshness + controller gates

→

Accepted or nonaccepted
Start, review, terminal records

# A reproducible offline workflow

Existing integration test: real subprocess, real Git, independent checks. No model call.




**Reproduce: python -m pytest tests/test_cli_integration.py -q**

Diagram summarizes assertions from the passing test. Ready for review is not acceptance.

# The first real problems

REPORTED INTEGRATION FAILURE · PR #1

## The worker wrote its completion marker to a file

Extra files violated scope.
One cleanup loop exhausted the tool budget.

REQUIREMENT CHANGE

Final response on stdout.
No completion files or tool emission.

REPRODUCED EVIDENCE DEFECT · PR #4

## A rename hid a forbidden deletion

**existing.txt**
Outside allowed scope

→

**renamed.txt**
Allowed destination

Name-only evidence omitted the source.

CORRECTION

--no-renames exposes both paths.
The controller rejects the violation.

# Adversarial scrutiny: success after expiry

Original assumption: an observed successful exit can end the monitor loop.

**CONTROLLED-CLOCK COUNTEREXAMPLE**




## A delayed monitor must classify this as timeout.

**CORRECTION** **Check expiry** → **Observe completion** → **Check expiry again**

Keep exit/output evidence. Block acceptance and further attempts on expired execution.

Validation: 9 regression cases cover timing, controller actions, and check expectations.

# Human authority and truthful evidence

01

## Authorize dispatch

Fresh START challenge
Publish start record
Bind the clean baseline

02

## Request an action

ACCEPT or REMEDIATE:
fresh challenge + rationale
STOP: rationale

03

## Record the outcome

Publish the request
Recheck freshness
Apply controller gates

**0 or 1**

Enabled remediation attempts
Fixed scope in the human CLI

**Saved request ≠ processed decision**

Unauthenticated records. No resume authority.

# What the evidence actually establishes

**575**

## Offline suite passed

Archived Windows documentation-validation run.
Synthetic workers, real Git, finite subprocesses, failure injection.

**123**

## Targeted AI-assisted review tests passed

PR #9 reports a separate final review pass.
Focused coverage, not a complete-suite rerun or human audit.

Historical experiment: PR #1 reports one Qwen eligibility-stage acceptance.

Raw run not inspected. No repeatability, causation, or full-feature acceptance claim.

No sandbox. Git-visible changes only. Caller-selected checks. Point-in-time freshness.

**Not established: production readiness, broad console/backend support, or semantic correctness.**

# Engineering contribution and lessons

OWNER-DIRECTED ENGINEERING

## Frame the problem. Challenge the evidence. Decide what to accept.

Matias set direction, requirements, review priorities, and acceptance decisions.

Extensive AI collaboration in architecture, implementation, investigation, testing, and review.

TRANSFERABLE LESSONS

**Evidence semantics shape what a gate can enforce.**

**Counterexamples belong in regression tests.**

**A narrower guarantee can be the better design.**

Possible next step: repeatable console and backend validation.